

[Home](#) > [My Courses](#) > [Portfolio Management](#) >[M8: Better Backtesting and Reinforcement Learning](#) > Lesson Notes

Lesson Notes

MODULE 8 | LESSON 2

COMBINATORIAL PURGED CROSS-VALIDATION

Reading Time	30 minutes
Prior Knowledge	"Walk-forward" backtesting, k-fold validation, Hyperparameters, Basic Python
Keywords	Combinatorial Purged Cross-validation

In the last lesson, we discussed the main approaches to backtesting as well as some of the most common flaws in its performance. We also discussed how backtesting can be misused leading to false investment strategies and how bias creeps into published research that is based on overtesting. We also discussed methods for detecting and controlling this potential problem. In this lesson, we continue our focus on backtesting by introducing a variation of k-fold validation that overcomes certain disadvantages associated with k-fold as well as Walk Forward optimization.

1. Walk Forward

In the last lesson, we defined the walk-forward (WF) approach to backtesting in detail. In brief, it involves testing the model on multiple, successive, out-of-sample periods from the historical data.

1.1 Advantages of Walk Forward

For many reasons, it seems like a natural and logical approach to backtesting:

1. **WF is realistic:** The strategy for time $t+1$ is determined using the data for all times up to and including t , which is similar to real-life decision-making where we have access to information about the present and past to make decisions about the future; in the language of mathematics or probability theory, we say that "history is a filtration" (de Prado 14).

2. **WF is easy to interpret:** WF test results straightforwardly present the performance of the strategy in the actual past, as opposed to a simulation or some highly stylized scenario. WF backtesting is indicative of how the strategy would have performed under circumstances exactly like those that actually occurred. Since those circumstances actually occurred, they represent plausible circumstances that might occur again.

1.2 Disadvantages of Walk Forward

On the other hand, those advantages can also qualify as disadvantages from a slightly different perspective.

1. While WF suggests how a strategy would have performed under exactly the set of circumstances that did occur, it is very unlikely that those exact circumstance will be repeated. While it may be reasonable to assume some serial correlation in time series in many situations, it can be dangerous to rely too heavily on this assumption. There is no guarantee that the future time series will resemble historical time series; in fact, there is practically a guarantee that the future will not mirror the past. Taking this perspective to the extreme, it is actually illogical to test a strategy using the one time series that will almost surely not occur—the time series that already occurred.
2. The other main disadvantage of WF is that there is only one historical time series, which suggests that only one complete backtest is possible. In a sense, only “a single scenario is tested” (de Prado 15). A strategy that performs well in a backtest that includes all relevant and available historical data could be overfit—and there is no out-of-sample time series data to check this.
3. The initial strategic decisions of the model are based on a very limited portion of the training data. At time $t=1$, the model must actually make a prediction and/or strategic decision based on no information. The solution to this problem is to allow for a burn-in period: Say the strategy only starts trading on $t=11$ or $t=101$ so that it has the benefit of 10 or 100 data points on which to base its prediction or trading decision. This solution comes at a cost: The test set is reduced by however many data points are allowed for the burn-in period. And even with this solution in place, the fact remains that the strategy benefits from almost all the data points only at the very end of the time series, that is, for only a small portion of the test set.
4. Leakage occurs when the above mentioned filtration is not perfect. For example, if there is serial correlation in the explanatory (feature) variables and dependent (label) variables. Take, for instance, a serially correlated explanatory variable X and the dependent variable Y . When we train a model on data through time t , and test the model starting with time $t+1$, the model has information about X_{t+1} due to the serial correlation: in the extreme, $X_t \approx X_{t+1}$ (de Prado 14–5). This leads to an artificial improvement in performance.

Of course, leakage can be managed by excluding from the test set those data points we consider to contain information from the training set. Conversely, we can exclude from the training set those data points we consider to have information about the testing set. This is called purging the training set data. Since leakage can be managed in this way, it does not disqualify the WF approach. However, it does unfortunately reduce the training set data somewhat.

2. Cross-Validation

2.1 Advantages of Cross-Validation

The main advantages of cross-validation (CV) correspond to the disadvantages of WF.

1. **Corresponding to WF disadvantage #1:** CV does not maintain the sequence of data, so testing set data points may correspond to historical dates prior to the training set data points. While CV still uses historical (and thus, realistic) data, it does not assume that the data points will occur in the same sequence that they did historically. Like the unknown future, the test data do not simply repeat the past. This should therefore lead to a more robust assessment of future performance (de Prado 16).

2. **Corresponding to WF disadvantage #3:** CV partitions the data into sets of equal size, so the model has been trained on the same amount of data before each test. As de Prado notes, “This makes outcomes comparable across periods, in terms of the amount of information used to make those decisions” (16).

3. **Corresponding to WF disadvantage #4:** Unlike WF with a burn-in period, each data point in CV is used to test the strategy. According to de Prado, “There is no warm up subset, thereby achieving the longest possible out of sample simulation” (16).

2.2 Disadvantages of Cross-Validation

The disadvantages of CV actually overlap with the disadvantages of WF—except for one:

1. In CV, each data point is only used in a single test set, just as in WF.
2. Leakage is possible in CV, as in WF. Whereas in WF the solution was purging, CV requires embargoing to ensure that information about the test data does not find its way into the training data. Similar and complementary to purging the time series, embargoing the time series entails discarding any training data that derive from historical data immediately following the testing data. This problem does not arise in WF because “the training set always predates the testing set” (de Prado 14).
3. Unlike WF, CV does not have straightforward interpretation. The performance assessment from a CV backtest does not indicate how the strategy would have performed under concretely historical circumstances.

3. Combinatorial Purged Cross-Validation

Combinatorial purged cross-validation (CPCV) is de Prado’s answer to the main disadvantage of both WF and CV, in that they only permit a data point to be used in a single test. With CPCV, each data point can be used many times. Take for example a dataset partitioned into six parts:

- In k-fold CV, there are six total tests where the test set is one-sixth of the complete time series.
- In CPCV, two of the six groups, or one-third of the data, can be used in each test (as in the figure below), and this leaves two-thirds of the data for training. CPCV makes it possible to incorporate each observation into five unique tests. Each observation could actually be used in ten total tests, but this would require that three groups (half the data) are used in each test and only one-half of the data is used for training. Since, generally, proper model training should ideally use more than half of the available data, this approach is not recommended.

Figure 1: Combinatorial Purged Cross-Validation with Six Folds and Two-Fold Test Sets

	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	Paths
G1	X	X	X	X	X											5
G2	X					X	X	X	X							5
G3		X				X				X	X	X				5
G4			X				X			X			X	X		5
G5				X				X			X		X		X	5
G6					X				X			X		X	X	5

Source: López de Prado, Marcos. "Advances in Financial Machine Learning: Lecture 5/10 (seminar slides)." 29 Sep 2018, p. 19. <https://ssrn.com/abstract=3257497> or <http://dx.doi.org/10.2139/ssrn.3257497>.

Note in the figure above that test sets immediately follow training sets and training sets immediately follow training sets, so there may still be the need for a data purge or embargo.

Given the higher number of backtests available through CPCV, we can better estimate the distribution of the Sharpe ratios, instead of a single, possibly overfit, estimate for the Sharpe ratio. And from this distribution, we can assign a confidence level to each Sharpe ratio. As Berger explains, "In general, the high number of paths leads to a more confident estimate of the true Sharpe ratio and leads to fewer false discoveries in contrast to WF and CV" (44). Indeed, a higher Sharpe ratio in which we have more confidence has been our goal in this and the last lesson.

Of course, we are not restricted to using the Sharpe ratio for our portfolio performance metric in any of the backtesting approaches we have discussed in this lesson. However, de Prado has provided a precise formula to describe the variance of the sample means of the Sharpe ratio (or the maximum expected value of the Sharpe ratio) given that we can assume its normality. In order to assign a confidence level to some other metric, you only need to know that it is also normally distributed or how to tractably define the density function of whatever distribution is more appropriate.

4. CPCV in Python

There are of course Python implementations available online. For example, the following snippet was taken from Mark Ivanov's Github, which uses CPCV to efficiently find a large number of hyperparameters for a given model:

```
def generate_cv_combinations():

a = list(range(1, NUMBER_OF_GROUPS+1, 1))

cv_custom_comb = []

test_groups = list(combinations(a, 2))

path_dict = dict()
```

```

for i in range(1, NUMBER_OF_GROUPS+1, 1):

    path_dict[i] = 1

    for g1, g2 in test_groups:

        out = (g1, path_dict[g1]), (g2, path_dict[g2])

        path_dict[g1] += 1

        path_dict[g2] += 1

    cv_custom_comb.append(out)

    return cv_custom_comb

```

(Ivanov lines 57–69)

Review the full `cpcv.py` file, understand how it accomplishes this objective using CPCV, and know what assumptions it makes.

5. Conclusion

In this lesson, we incorporated what we learned in the last lesson about the distribution of performance metrics, namely the Sharpe ratio. We also described a backtesting approach that provides more backtests than WF or CV so that we can arrive at performance metrics to which we can assign higher confidence levels.

References

- Berger, Thomas. "Dynamic Investment Strategies with Machine Learning Methods." 5 Jun 2019, Ludwig-Maximilians-Universität München Institut für Statistik, Master's thesis. https://epub.ub.uni-muenchen.de/69183/1/MA_BergerThomas.pdf.
- Ivanov, Mark. "fast_combinatorial_cv." *GitHub*. https://github.com/markmipt/fast_combinatorial_cv. Code shared here is under the [Apache license 2.0](#)
- López de Prado, Marcos. "Advances in Financial Machine Learning: Lecture 5/10 (seminar slides)." 29 Sep 2018. <https://ssrn.com/abstract=3257497> or <http://dx.doi.org/10.2139/ssrn.3257497>.